

RiSD

WYKŁAD V

Dziel i zwyciężaj, cz. 2

Problem (Para najbliższych punktów)

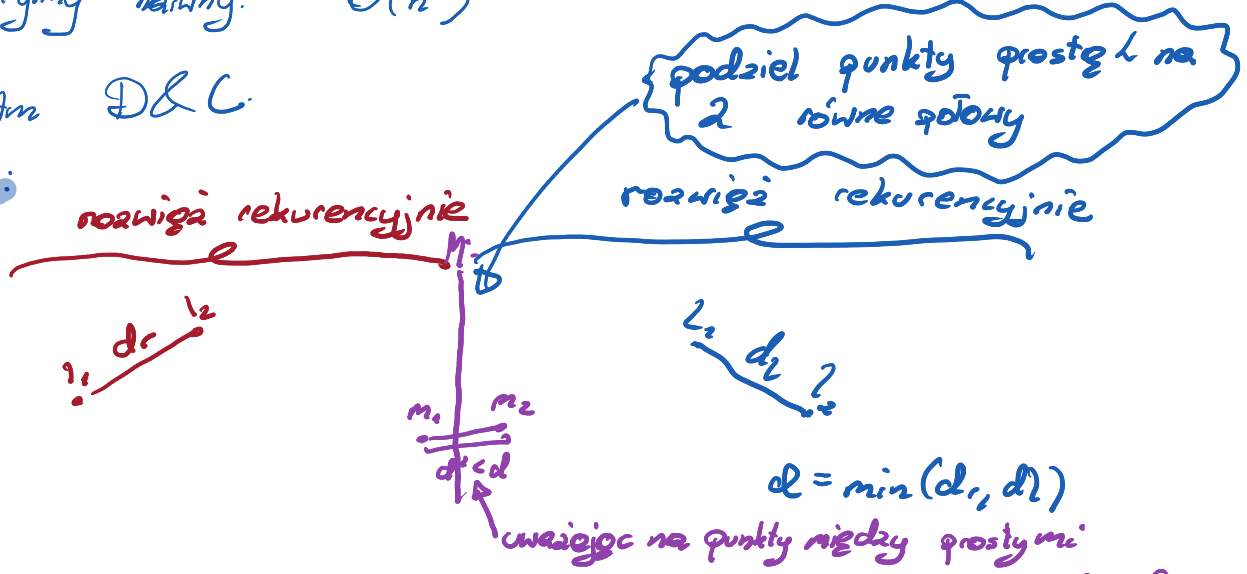
Dane $P = \{ \overbrace{(x_1, y_1)}^{p_1}, \dots, \overbrace{(x_n, y_n)}^{p_n} \}$

Zadanie: Znaleźć takie i oraz takie j , że odległość między p_i oraz p_j jest najmniejsza

Algorytm naiwny: $\Theta(n^2)$

Algorytm D&C.

Idea:



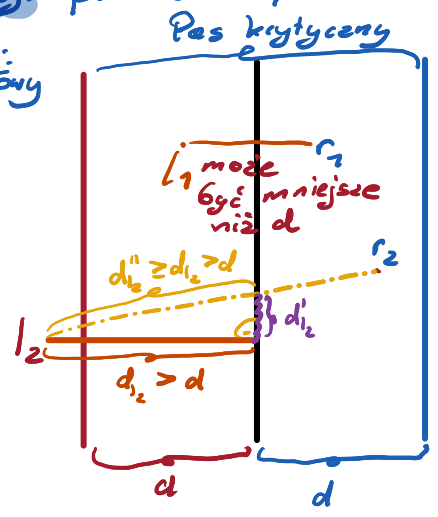
Główny problem - jak efektywnie radzić sobie z punktami położonymi z dwóch stron M , których dystans jest mniejszy niż d ?

Obserwacja 1°:

Niech L i R będą prostymi równoległymi do M oddalonymi (odpowiednio na lewo i prawo od M) o d . Wtedy wszystkie takie punkty (które są w d) będą pomiędzy nimi. Przestrzeń między L i R nazwiemy pasem krytycznym.

Inaczej: jest sens sprawdzać tylko punkty odległe od M o co najwyżej d .

$d-d$: rysunek



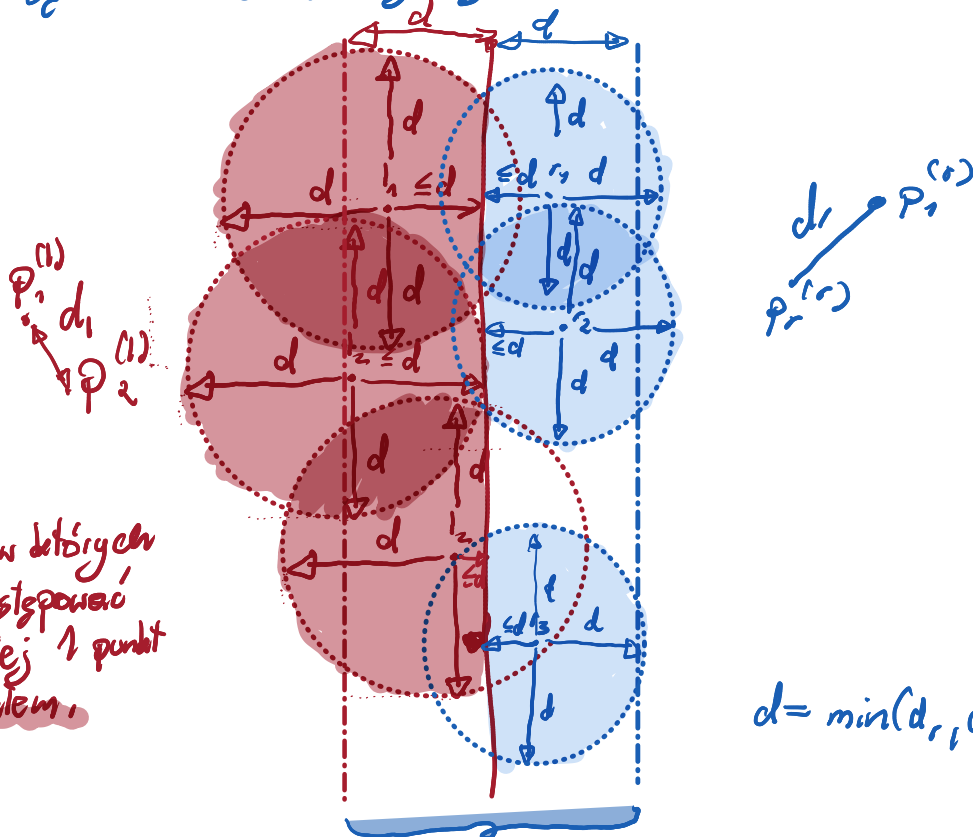
Obserwacja 2° -

Niech Y zawiera wszystkie punkty z pasa krytycznego posortowane rosnąco po y .

Wtedy najbliższe parę z Y można znaleźć, porównując każde p_i z następnymi punktami po p_i .

Zamiast dowodu przedstawie rozwiązanie prowadzące do tej obserwacji. Są dość długie, dlatego odsyłam do sekcji „ostatnie demistyfikacje”, jeśli czytelnik będzie potrzebował rezultatu od razu.

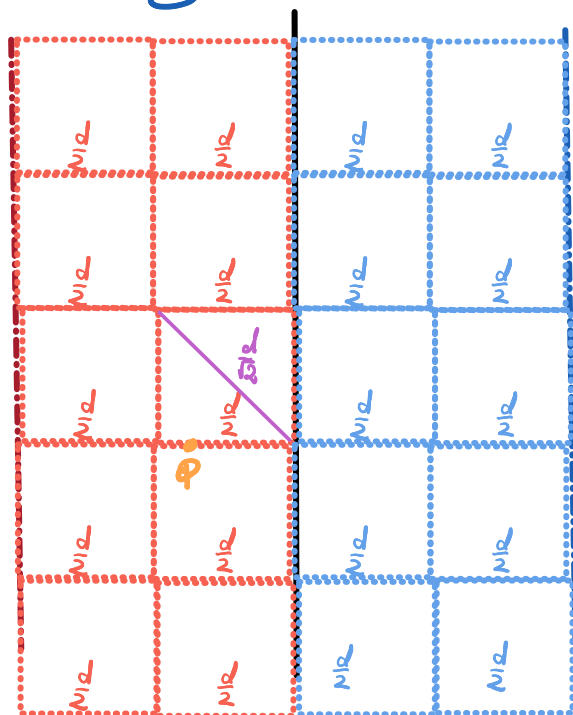
Narysujmy koła o promieniu $d = \min(d_1, d_2)$, wokół każdego punktu z pasa krytycznego. Zauważmy, że w każdym z tych koł nie mogą wystąpić (nie licząc ewentualnie obrotu) 2 punkty z tej samej połowy, bo to by znaczyło, że są oddalone o mniej niż d (w szczególności mniej niż d_1, d_2), a to jest sprzeczność bo rekurencyjnie stwierdziliśmy, że punkty z najbliższej pary punktów na każdej z połów są oddalone co najmniej o d .



Obszary - w których może występować co najwyżej 1 punkt zacieniowane.

$$d = \min(d_1, d_2)$$

Zmienimy nieco myślenie - założymy, że chcemy dla pewnego p z tego pasa (bądź p jest a lewej połowie) znaleźć najbliższy punkt w drugiej (bądź prawej) połowie. Podzielimy pas na kwadraty o boku $\frac{d}{2}$ tak, aby p znalazł się na jednej z dolnych podstaw



Zauważmy, że każdy kwadrat zawiera co najwyżej 1 punkt.

Jest tak, że 2 punkty w kwadracie są oddalone co najwyżej o $\frac{d\sqrt{2}}{2} = \frac{d}{\sqrt{2}}$ (tyle ile ugnosi przekątna), ale $\frac{d}{\sqrt{2}} < d$, więc byłyby oddalone od siebie o mniej niż d .

To nam już daje pewną intuicję, jak należy postąpić - sprawdzimy kolejne punkty na tym pasie (malejaco po y) i wystarczy wtedy porównywać punkt z kwadratem po jednej stronie - a punktami ze stałą ilości kwadratów po stronie drugiej - pytanie KU?

The diagram shows a 2D grid divided into two regions by a diagonal line. The left region is outlined with red dashed lines and contains labels $N_{1/2}$ and a yellow d . The right region is outlined with blue dotted lines and contains labels $N_{1/2}$ and a yellow d' . A diagonal line runs from the bottom-left to the top-right, labeled Z_d and $Z_{d'}$. The grid is used to illustrate the construction of a 2D convolution operation.

Ale to trochę szliskie... jak
od znalezć te 8 punktów? Poza tym
8 sprawdzeń per element to trochę
dużo. Czy da się lepiej? **TAK**

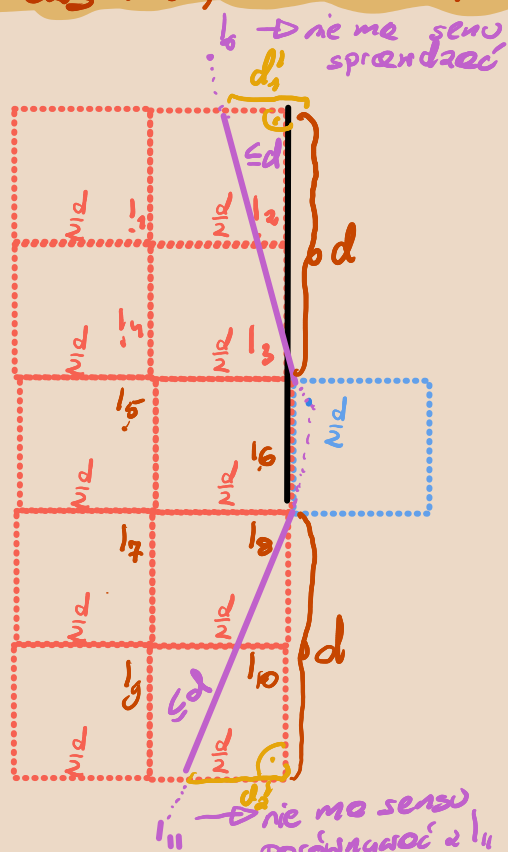
ΘΣΤΑΤΕΣΤΩΝ ΔΕΜΙΣΤΥΦΙΚΑΛΩ

Jeżeli tu są punkty d
bliższe p_1 niż $p_1, \dots, p_4$
zostaną one zalezione
w poprzednich iteracjach
po y_i

Punkty, których odległości jest sens
rozważać są 4 kwadrantów, więc
jest ich 4.

Co prawda porównanie p z p_4, p_6
 p_3 będzie bezsensowne (bo są w tej
 samej potęgach, a więc odległość między
 nimi będzie $\geq d$), ale i tak jest lepiej
 niż wyżej

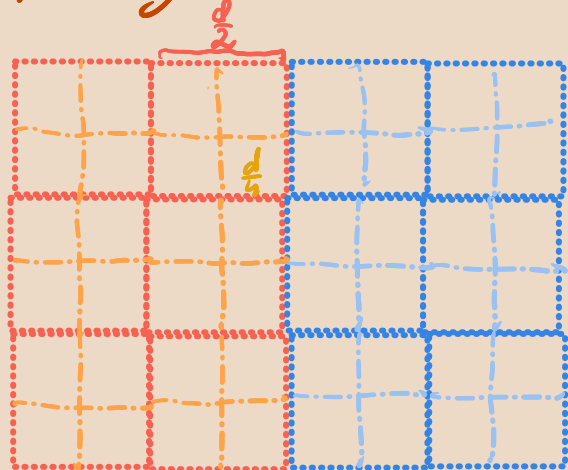
Aneks (ku przestródze) - jak tego NIE rozkurmniać (albo jak to robić przez MNOŻĄ I PRZEGRYWAJ, a nie D&C)



Stąd wychodzi maksymalnie 10 sprawdzeń
Ale coś tu nie gra - na pewno da się lepiej!

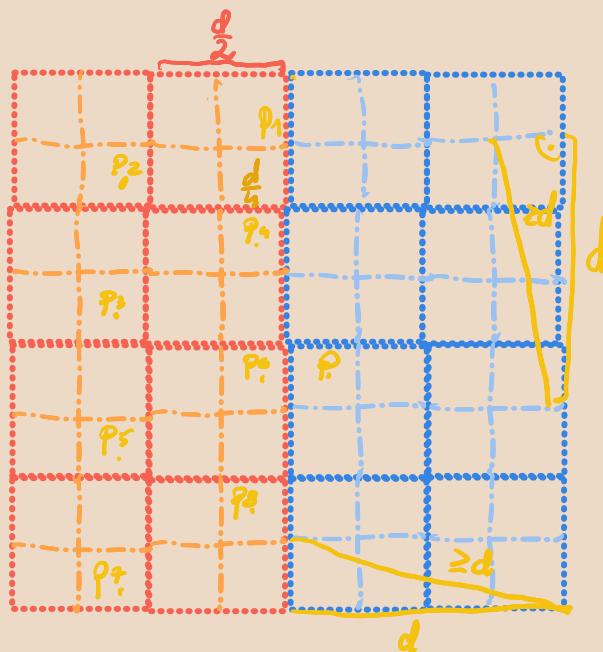
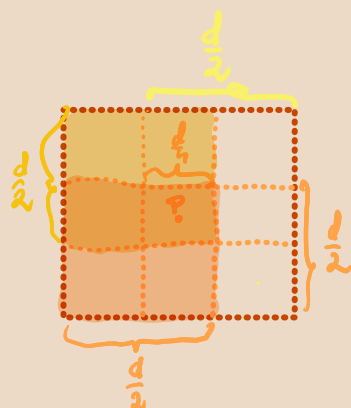
Pracicz dalej, że l_1 i l_2 są oddalone o zbyt małą odległość.

Podzielmy te kwadraty jeszcze na mniejsze o boku $\frac{d}{4}$.



Wtedy:

- każdy kwadrat o boku $\frac{d}{4}$ zawiera co najwyżej 1 punkt (bo $\frac{d\sqrt{2}}{4} = \frac{d}{2\sqrt{2}} < d$)
- z rezultatu wyżej na każdy kwadrat o boku $\frac{d}{2}$ przypadając będzie co najwyżej jeden wypełniający go o boku $\frac{d}{4}$, który zawiera punkt
- ⇒ jeżeli kwadrat o boku $\frac{d}{4}$ zawiera punkt - sąsiadnie z tej samej połowy nie zawierają.



$$\frac{d}{2}\sqrt{2} \geq d$$

To jeszcze bardziej komplikuje :-)

Mamy już sposób na pas krytyczny, ale jak wygląda optymalny algorytm?

Preprocessing:

$T_x \leftarrow$ punkty z P posortowane po x

$T_y \leftarrow$ ——— " ——— y

Początek algorytmu

1. $M \leftarrow$ prosta dzieląca P na pół (środek T_x)

$P_L \leftarrow$ punkty na lewo od M

$P_R \leftarrow$ punkty na prawo od M

$\Theta(n)$ $\left\{ \begin{array}{l} T_x^{(R)}, T_y^{(R)} \leftarrow \text{posortowane po } x, y \text{ tablice punktów z } P_R \text{ (stworzone z } T_x, T_y) \\ T_x^{(L)}, T_y^{(L)} \leftarrow \text{————— " ————— } P_L \text{ (————— " —————)} \end{array} \right.$

2. Rekurencja:

$(l_1, l_2) \leftarrow$ rozwiązanie dla P_L o odległości d_L

$(r_1, r_2) \leftarrow$ ——— " ——— P_R ——— " ——— d_R

$d \leftarrow \min(d_L, d_R)$

Y' $\leftarrow$ tablica punktów a pasu krytycznego

$\Theta(n)$ $\left\{ \begin{array}{l} \text{for } i = 1 \text{ do } |Y'|: \\ \quad \text{dla } Y'[i] \text{ sprawdź odległość między nim a punktami} \\ \quad Y'[i+1], \dots, Y'[i+7] \text{ i zapisz parę } (Y'[i], Y'[i+j]) \text{ } 1 \leq j \leq 7 \\ \quad \text{o odległości } < d \text{ jeśli istnieje} \end{array} \right.$

TC:

Koszt $T(n)$:

Sortowanie — $\Theta(n \log n)$

Rekurencja — $2 \cdot T(\frac{n}{2})$

Iteracja po pasie krytycznym — $\Theta(n)$

$T(n) = 2T(\frac{n}{2}) + \Theta(n \log n) = \Theta(n \log^2 n) \rightarrow$ jeśli sortujemy $T_x^{(R)}, \dots, T_y^{(L)}$ z każdym wywołaniem rekurencyjnym.



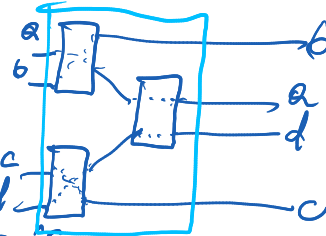
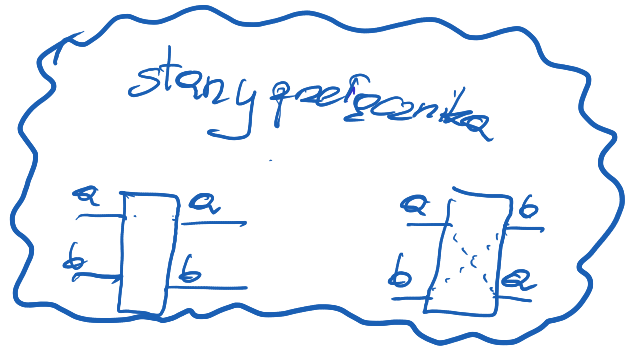
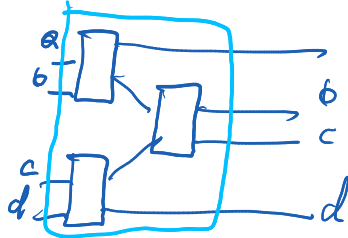
Z preprocessingem $\Theta(n \log n) + T(n) = \Theta(n \log^2 n)$
(sortowaniem na początku)

Sieci przełączników



druty

głębokość: 2
rozmiar: 3



PERMUTACJA WYJŚĆ

Najważniejsze parametry sieci:

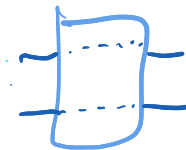
~ głębokość sieci (max przełączników na ścieżce do wyjścia)

~ rozmiar sieci (ilość przełączników)

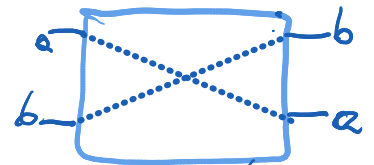
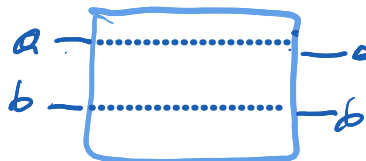
Łączenie $n = 2^k$

cel: Skonstruować „dobrą” sieć, która umożliwia otrzymanie dowolnej permutacji wyjść

$n = 1$

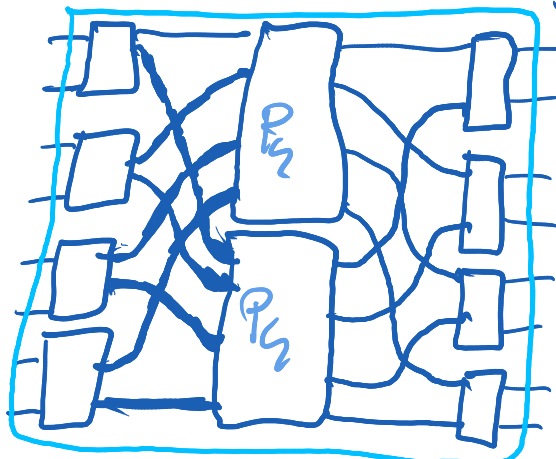


$n = 2$:

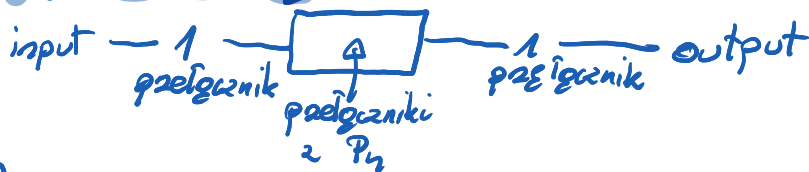


(sieć Beneša-Waksmana)

P_3 :



Głębokość P_n

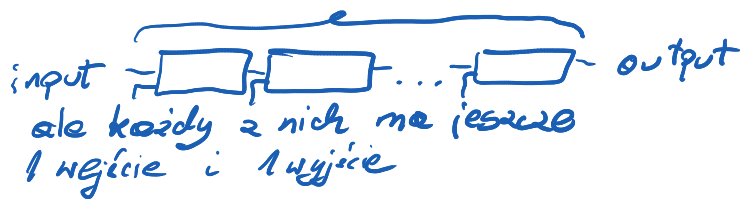


$$G(n) = \begin{cases} 1 & n \leq 2 \\ 2 + G(\frac{n}{2}) & n > 2 \end{cases}$$

$$\Rightarrow G(n) = 2 \log n - 1$$

Rozmiar P_n

$G(n)$ przełączników



$$\Rightarrow S(n) = \frac{n \cdot G(n)}{2} = n \log n - \frac{n}{2}$$

Zastosowania: ???

Niskopoziomowa implementacja itertools ???

TW

Sieć Beneša - Waksmana umożliwia otrzymanie każdej permutacji.

d-d:

Niech π będzie dowolną permutacją. Rozmy my graf

$G_\pi = (V, E)$ następująco:

$$V = V_1 \cup V_m \cup V_0$$

Wejścia sieci

1
2
3
⋮
n

Reszta, czyli środek sieci (symbolizacja)

1'
2'
⋮
n'

Wyjścia sieci

1''
2''
⋮
n''

" granica port ma efektywność"

$$E = E_1 \cup E_m \cup E_0$$

$\frac{n}{2}$ krawędzi łączące sąsiadów $i, (i+1)$

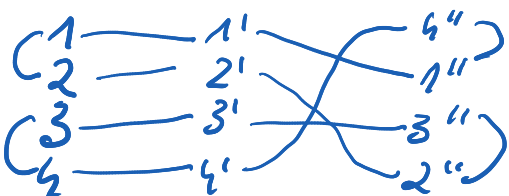
$2n$ krawędzi łączące $i - i' - i''$

$\frac{n}{2}$ krawędzi łączące sąsiadów i, i''



Przykład

$$G_\pi \text{ dla } 1234 \xrightarrow{\pi} 4132$$



Każdy i, i', i'' ma stopień 2.
 • i - ma sąsiadów $i'+1$ oraz i'
 • i' - ma sąsiadów i oraz i''
 • i'' - ma sąsiadów i' oraz $i!$

Teraz: G_π to suma rozłącznych cykli o parzystej długości.

(1) tylko cykle:

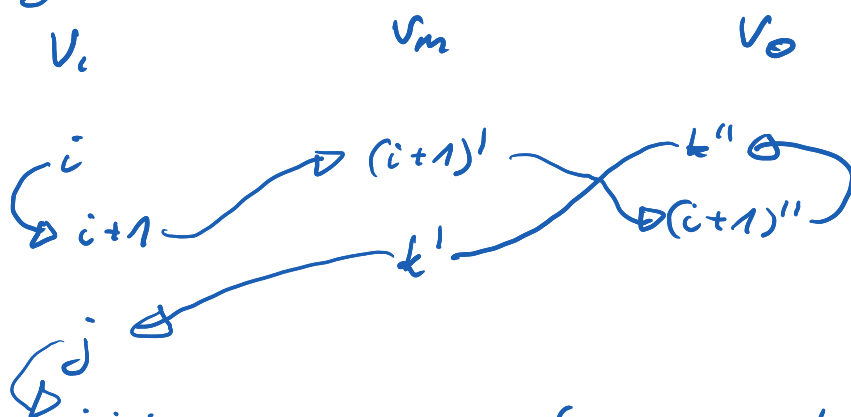
Niech G_π zawiera ścieżkę, wtedy niedzi będzie jej początkiem. i ma stopień 2, wtedy do tej ścieżki można dołączyć drugą (od 2-giej krawędzi i) wtedy i przestaje być początkiem ścieżki

(2) rozłączone:

Niech i będzie dowolnym wspólnym wierzchołkiem dwóch cykli. i ma stopień 2. Czyli jedna krawędź wychodząca - druga wychodząca, zatem element poprzedzający (i następujący) po i (zgodnie z ruchem wskazówek zegara) będzie taki sam w obu permutacjach

(3) pozostałe:

Jak wygląda cykl?



Mozna w nim wyróżnić pewne etapy, każdy o krociach:

1° z i idziemy do $i+1$

2° z $i+1$ do $(i+1)'$ → przekształcamy do V_m

3° z $(i+1)'$ do $(i+1)''$ → — — — V_0

4° z $(i+1)''$ na jakiś k''

5° z k'' do k' → — — — V_m

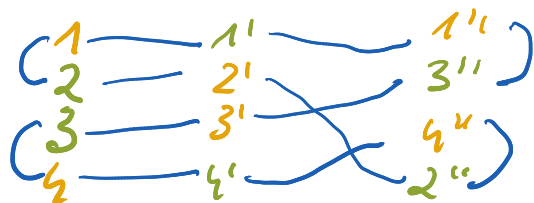
6° z k' na j → wracamy do V_m

Czyli każdy etap to wędrowanie do V_m, V_0 , później powrót do V_m, V_i w 6 krokach. Ostatni krok to będzie $i' \rightarrow i$ (6°)

Zatem cyklicznie wykonamy 6k kroków → a szczególnie pozostałe liczby.

W sumie policzliśmy krawędzie, ale w cyklu ich liczba jest taka sama jak wierzchołków.

Kolorunki :-)



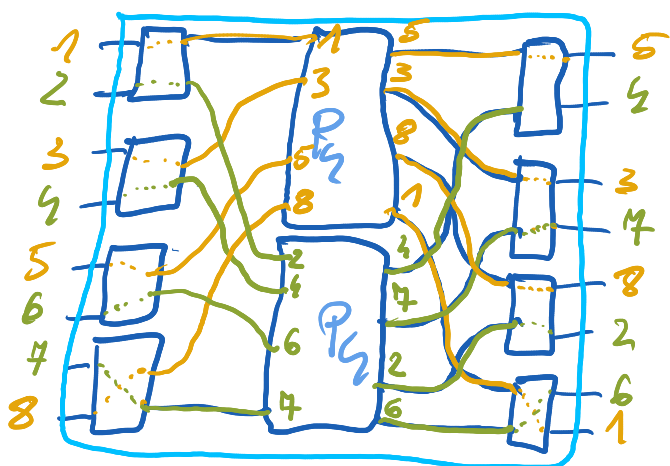
Dla każdego cyklu:

- Wiedza o pozostałych indeksach kolorujemy na zielono, o pozostałych na żółto, cykl ma pozostałą długość, więc nie będzie 2 wiadomości o tym samym kolorze

Zauważmy, że w każdym fragmencie:

- i, i', i'' , czy też i'', i', i' , i, i'' - będą w tym samym kolorze, a i' w innym
- jeżeli i'', k'' sąsiadują to cykl przejdzie $i'' \rightarrow k''$ lub k'', i'' i kolor k'' będzie inny, niż i''
- $i, i+1 \rightarrow$ uwaga, jak wyżej.

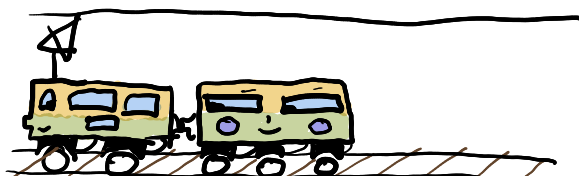
Stąd ostatecznie tak osiągnemy:



1°

Góra na Pienin (cegły żółte do górnego P_1)

z zielone na DoT na Kolornice (dolne P_1)



Bp 2026.

2° W każdym P_k permutujemy wejścia, tak, aby każdy trafił do docelowego przełącznika, da się tak zrobić, gdyż żadne 2 i, j nie będą rywalizować o ten sam przełącznik. Złożymy przecięnie - niech i idzie na k -te wyjście a j na $k+1$. Wtedy $k=i$ $k+1=j$ mają różne kolory, zatem i, j także, a to sprzeczność, bo wtedy trafiłyby do różnych P_k .

3° każde wejście trafi do danego przełącznika teraz pozostaje tylko odpowiednio go ustawić.